📘 Agentic架构 9: 思维树（Tree-of-Thoughts）

> **思维树（Tree-of-Thoughts，ToT）** —— 将智能体的问题解决能力**从线性思维链提升为多路径的探索式搜索**。 思维树智能体不再生成单一的、顺序的推理链条，而是在问题的每个阶段生成**多个候选“思维”或下一步行动**。随后，它对这些思维进行**评估**，剪除无效或前景不佳的分支，并**拓展最有希望的分支**。这样就形成了一个**搜索树**，智能体可以在其中**回溯、探索替代方案**，并系统地导航复杂的问题空间。

定义

思维树（Tree-of-Thoughts, ToT） 是一种智能体推理框架，其**将问题解决建模为对一棵树的搜索**。智能体同时**探索多条推理路径（分支）**。在每一步，它生成多个潜在的**下一步（“思维”）**，评估其可行性，并决定继续探索哪些路径，从而有效地对搜索空间进行**剪枝**。

高级工作流程

1. **分解：** 将问题分解为一系列**步骤或思维**。
2. **思维生成：** 针对问题的当前状态，智能体生成**多个潜在的下一步或思维**。这就在搜索树中创建了**分支**。
3. **状态评估：** 每个新思维（通向一个新状态）由一个 **“评判器”或验证函数**进行评估。评估可以涉及：
   - **有效性：** 此步骤是否符合问题规则？
   - **进展：** 此步骤是否让我们更接近解决方案？
   - **启发式判断：** 此路径是否可能成功？
4. **剪枝与拓展：** **无效或前景不佳的分支被剪除**。随后，智能体从最有前景的**活跃分支**出发，重复思维生成过程。
5. **解决方案：** 该过程持续进行，直到达到**目标状态**。解决方案即为**从根节点到目标节点的思维路径**。

适用场景 / 应用

- **逻辑谜题与数学问题：** 具有明确规则和目标状态，需要**多步骤、非线性推理**的问题（例如数独、过河问题）。
- **复杂规划：** 当任务需要**详细计划**，且操作顺序至关重要、必须遵守**约束条件**时（例如规划包含多段行程和预算限制的复杂旅程）。
- **创意写作或代码生成：** 在确定最终方案前，**探索多个**故事分支或实现策略。

优势与劣势

- **优势：**
  - **鲁棒性：** **系统地探索问题空间**，与单次通过的方法相比，更不容易卡住或产生错误答案。
  - **处理组合复杂性：** 非常适合**可能序列数量庞大**的问题。
- **劣势：**
  - **计算开销大：** 与简单的思维链提示相比，需要**显著更多**的大语言模型调用和状态管理，因此**速度更慢、成本更高**。
  - **需要良好的评判器：** 搜索的有效性在很大程度上取决于**状态评估逻辑的质量**。

阶段 0：基础与环境搭建

安装所需的库，并配置 API 密钥。

```
pip install -q -U langchain-nebius langchain langgraph rich python-dotenv
```

```
"""
思维树架构（Tree-of-Thoughts, ToT）- 多路径推理与自我评估的智能体系统
思维树架构是一种高级推理框架，其核心思想是：
- 将复杂问题的求解过程建模为树的搜索
- 在每个决策点生成多个候选思维路径
- 通过自我评估机制对每条路径进行评分
- 选择最优路径继续探索，或回溯重新考虑
"""

import os
import re
from typing import List, Dict, Any, Optional
from dotenv import load_dotenv
from collections import defaultdict

# ==================== Pydantic 组件导入 ====================
# Pydantic用于数据验证、配置管理和结构化输出定义
from pydantic import BaseModel, Field

# ==================== LangChain 组件导入 ====================
# LangChain提供LLM抽象和提示模板等核心功能
from langchain_nebius import ChatNebius                # Nebius云平台的大语言
模型接口
from langchain_core.prompts import ChatPromptTemplate  # 结构化提示模板

# ==================== LangGraph 组件导入 ====================
# LangGraph提供基于图的状态机框架，用于构建复杂的推理流程
from langgraph.graph import StateGraph, END          # 状态图核心组件
from typing_extensions import TypedDict              # 类型字典定义

# ==================== 可视化与调试工具 ====================
# Rich库提供增强的终端输出格式，便于调试和演示
from rich.console import Console                     # 富文本控制台输出
from rich.markdown import Markdown                   # Markdown格式渲染
from rich.tree import Tree                           # 树形结构可视化

# ==================== API密钥与环境配置 ====================
"""
思维树架构环境变量配置说明：
- NEBIUS_API_KEY: Nebius LLM服务的认证密钥
- LANGCHAIN_API_KEY: LangChain追踪服务的认证密钥
- LANGCHAIN_PROJECT: LangChain追踪项目标识符（思维树专用）
思维树架构主要依赖LLM进行推理和评估，不需要外部向量存储或图数据库
"""

load_dotenv()  # 从.env文件加载环境变量
```

```
# 配置LangChain追踪系统，用于监控和调试思维树推理过程
os.environ["LANGCHAIN_TRACING_V2"] = "true"                    # 启用V2版本
追踪
os.environ["LANGCHAIN_PROJECT"] = "Agentic Architecture - Tree-of-Thoughts
(Nebius)"  # 项目标识

# 验证关键环境变量是否存在
required_vars = [
    "NEBIUS_API_KEY",       # Nebius LLM服务密钥
    "LANGCHAIN_API_KEY"     # LangChain追踪服务密钥
]

for var in required_vars:
    if var not in os.environ:
        print(f"Warning: Environment variable {var} not set.")

print("Environment variables loaded and tracing is set up.")
```

阶段 1：定义问题环境

思维树系统需要一个**定义良好的环境**来运行。针对狼、山羊和卷心菜谜题，这意味着需要通过编程定义以下内容：

1. **状态表示：** 一种描述所有物品所在**位置**的方法。
2. **验证规则：** 一个用于**检查状态是否无效**的函数（例如，山羊和卷心菜被单独留在一起）。
3. **目标状态：** 一种**检查谜题是否已解决**的方法。
4. **可行移动：** 一个用于确定给定状态下所有**合法移动**的函数。

```
# ==================== 控制台初始化 ====================
# 初始化Rich控制台实例，用于增强终端输出格式
console = Console()

# ==================== 狼-羊-白菜谜题状态定义 ====================
# 该谜题是一个经典的约束满足问题，要求将三个物品从河左岸运到右岸
# 约束条件：
# - 狼和羊不能单独留在一起（狼会吃羊）
# - 羊和白菜不能单独留在一起（羊会吃白菜）
# - 船只能承载农夫加最多一个物品
# - 农夫必须划船过河
class PuzzleState(BaseModel):
    """
    狼-羊-白菜谜题状态模型
    状态表示：
    - 左岸集合：当前在左岸的物品（初始包含所有物品）
    - 右岸集合：当前在右岸的物品（初始为空）
    - 船位置：当前船在左岸还是右岸
    - 移动描述：记录上一步移动的动作，用于可解释性

    属性:left_bank: 左岸上的物品集合（初始为{"wolf", "goat", "cabbage"}）
```

```
        right_bank: 右岸上的物品集合（初始为空集）
        boat_location: 船的位置，"left"或"right"
        move_description: 上一步移动的文字描述
    """
    # 使用集合存储物品，因为物品无序且唯一
    left_bank: set[str] = Field(default_factory=lambda: {"wolf", "goat",
"cabbage"})
    right_bank: set[str] = Field(default_factory=set)
    boat_location: str = "left"
    move_description: str = "Initial state."

    def is_valid(self) -> bool:
        """
        检查当前状态是否有效（没有物品被吃掉）
        有效条件：当船在左（右）岸时，右（左）岸不能同时存在狼和羊，或羊和白菜
        原理：农夫不在的河岸上，不能留下会互相伤害的物品组合

        返回:bool: True表示状态有效，False表示违反了约束
        """
        # 检查左岸
        if self.boat_location == "right":
            # 狼和羊不能单独留在左岸
            if "wolf" in self.left_bank and "goat" in self.left_bank:
                return False
            # 羊和白菜不能单独留在左岸
            if "goat" in self.left_bank and "cabbage" in self.left_bank:
                return False
        # 检查右岸
        if self.boat_location == "left":
            # 狼和羊不能单独留在右岸
            if "wolf" in self.right_bank and "goat" in self.right_bank:
                return False
            # 羊和白菜不能单独留在右岸
            if "goat" in self.right_bank and "cabbage" in self.right_bank:
                return False
        return True

    def is_goal(self) -> bool:
        """
        检查是否达到目标状态
        返回:bool: True表示所有物品都在右岸
        """
        return self.right_bank == {"wolf", "goat", "cabbage"}

    def __hash__(self):
        """
        哈希函数，用于将状态作为字典键或集合元素
        将可变集合转换为不可变集合(frozenset)，使状态可哈希，用于visited集合的重复状
态检测
        返回:int: 状态的哈希值
        """
        return hash((frozenset(self.left_bank), frozenset(self.right_bank),
self.boat_location))
```

```python
    def __eq__(self, other):
        """
        相等性比较，用于状态去重
        返回:bool: 两个状态是否等价
        """
        return self.__hash__() == other.__hash__()

# ==================== 移动生成函数 ====================
def get_possible_moves(state: PuzzleState) -> list[PuzzleState]:
    """
    从当前状态生成所有可能的合法下一状态
    移动规则:
    1. 农夫可以带一个物品过河（如果该物品在当前船所在的河岸）
    2. 农夫可以空手过河（船不带任何物品）
    生成的每个状态都会经过有效性检查，只返回合法状态

    参数:state: 当前谜题状态
    返回:list[PuzzleState]: 所有合法下一状态的列表
    """
    moves = []

    # 确定当前船所在的河岸上的物品集合
    current_bank = state.left_bank if state.boat_location == "left" else
state.right_bank

    # ==================== 移动选项1：带一个物品过河 ====================
    # 遍历当前河岸上的所有物品
    for item in current_bank:
        # 深拷贝当前状态，避免修改原状态
        new_state = state.model_copy(deep=True)

        # 根据船位置决定移动方向
        if state.boat_location == "left":
            # 从左岸移动到右岸：从左边移除，添加到右边
            new_state.left_bank.remove(item)
            new_state.right_bank.add(item)
            new_state.boat_location = "right"
            new_state.move_description = f"Move {item} to the right bank."
        else:
            # 从右岸移动到左岸：从右边移除，添加到左边
            new_state.right_bank.remove(item)
            new_state.left_bank.add(item)
            new_state.boat_location = "left"
            new_state.move_description = f"Move {item} to the left bank."

        # 只保留有效状态（不违反约束）
        if new_state.is_valid():
            moves.append(new_state)

    # ==================== 移动选项2：空手过河 ====================
    # 农夫单独划船过河，不带任何物品
    empty_move_state = state.model_copy(deep=True)

    if state.boat_location == "left":
```

```
        empty_move_state.boat_location = "right"
        empty_move_state.move_description = "Move the boat empty to the
right bank."
    else:
        empty_move_state.boat_location = "left"
        empty_move_state.move_description = "Move the boat empty to the
left bank."

    # 检查空手移动后的状态是否有效
    if empty_move_state.is_valid():
        moves.append(empty_move_state)

    return moves

print("Puzzle environment defined successfully.")
```

阶段 2：使用 LangGraph 实现思维树智能体

构建智能体本身。**工作流图的状态将追踪思维树中所有活跃的路径（分支）**。各个节点将执行思维树的关键操作：

1. **扩展路径（思维生成器）：** 一个由**大语言模型驱动**的节点，它查看每条活跃路径的**最后一个状态**，并从所有**合法可能性**中提出一个**有前景**的下一步移动。
2. **剪枝路径（状态评估器）：** 该节点在生成后进行清理。它将**移除任何进入无效状态或循环状态**（重复访问之前的状态）的路径。
3. **检查解决方案（目标检查）：** 一个条件节点，检查是否有任何**活跃路径已达到目标状态**。如果是，则终止循环。

```
# ==================== LLM初始化 ====================
# 使用Mixtral 8x22B模型进行思维树推理
# temperature=0.4: 适度随机性，允许生成多样化的候选思维
llm = ChatNebius(model="mistralai/Mixtral-8x22B-Instruct-v0.1",
temperature=0.4)

# ==================== 移动选择Pydantic模型 ====================
# 定义LLM选择最佳移动的结构化输出格式
class MoveChoice(BaseModel):
    """
    移动选择模型 - 思维树中的决策点
    该模型用于LLM从多个候选移动中选择最有希望的一个：
    - best_move_index: 选择的移动在候选列表中的索引
    - reasoning: 选择该移动的理由（增强可解释性）

    在完整的ToT架构中，这个模型可以用于引导搜索方向，但在当前简化实现中，我们采用广度优先
探索所有可能移动。
    """
    best_move_index: int = Field(description="The index of the best move
from the provided list of possible moves.")
    reasoning: str = Field(description="Brief reasoning for why this is the
most promising move.")
```

```python
# ==================== 思维树状态定义 ====================
# 定义ToT架构的状态数据结构
class ToTState(TypedDict):
    """
    思维树状态数据结构
    该状态管理整个树搜索过程:
    - active_paths: 当前正在探索的所有路径（树的叶子节点）
    - solution: 找到的解（完整的状态序列）
    搜索策略:
    - 采用广度优先搜索(BFS)思想，同时探索多条路径
    - 每条路径是一个状态列表，记录从初始状态到当前状态的历史
    """
    problem_description: str                        # 问题描述（如谜题说明）
    active_paths: List[List[PuzzleState]]           # 活跃路径列表（每条路径是
一系列状态）
    solution: Optional[List[PuzzleState]]           # 找到的解（完整路径）

# ==================== 搜索初始化节点 ====================
def initialize_search(state: ToTState) -> Dict[str, Any]:
    """
    搜索初始化节点 - 建立搜索树的根节点
    创建初始谜题状态作为搜索的起点，每条路径初始只包含初始状态

    参数:state: 当前ToT状态（此时为空）
    返回:dict: 包含初始路径列表的状态更新
    """
    # 创建初始谜题状态（所有物品在左岸，船在左岸）
    initial_puzzle_state = PuzzleState()
    return {"active_paths": [[initial_puzzle_state]]}

# ==================== 路径扩展节点 ====================
# 这是ToT架构的"思维生成器"(Thought Generator)
# 负责从当前路径生成所有可能的后续状态
def expand_paths(state: ToTState) -> Dict[str, Any]:
    """
    路径扩展节点 - 生成所有可能的下一状态
    该节点实现ToT的"分支"机制:
    1. 遍历当前所有活跃路径
    2. 对每条路径的最后状态，生成所有合法移动
    3. 每个移动创建一条新的扩展路径

    当前实现是广度优先的完整探索：不限制分支因子，探索所有可能移动，演示完整的树结构
    更高级的实现可以使用LLM的MoveChoice来选择最有希望的k条路径，实现"束搜索"(Beam
Search)策略。

    参数:state: 当前ToT状态，包含所有活跃路径
    返回:dict: 包含扩展后所有路径的状态更新
    """
    console.print("--- Expanding Paths ---")

    # 配置结构化LLM用于移动选择（本实现暂未使用）
    choice_llm = llm.with_structured_output(MoveChoice)
```

```
    # 构建移动选择提示模板（保留用于未来增强）
    prompt = ChatPromptTemplate.from_messages([
        ("system", "You are an expert logic puzzle solver. Your goal is to
solve the Wolf, Goat, and Cabbage problem. Analyze the current path and
choose the most promising next move from the list of options to reach the
goal."),
        ("human", "Problem: {problem}\n\nCurrent Path
History:\n{path_history}\n\nFrom the final state, choose the best next move
from this list:\n{possible_moves}")
    ])

    new_paths = []

    # 遍历所有活跃路径
    for path in state['active_paths']:
        last_state = path[-1]                                    # 当前路径
的最后一个状态
        possible_next_states = get_possible_moves(last_state)    # 生成所有
可能的下一状态

        # 如果无法继续移动，跳过该路径（死路）
        if not possible_next_states:
            continue

        # 构建路径历史描述（用于可解释性）
        path_history_str = " -> ".join([s.move_description for s in path])
        possible_moves_str = "\n".join([f"{i}: {s.move_description}" for i,
s in enumerate(possible_next_states)])

        # 当前实现：探索所有可能的移动（完整广度优先）
        # 这展示了ToT的完整树结构，但可能导致指数级增长
        for next_state in possible_next_states:
            new_paths.append(path + [next_state])

    console.print(f"[cyan]Expanded to {len(new_paths)} potential paths.
[/cyan]")
    return {"active_paths": new_paths}

# ==================== 路径剪枝节点 ====================
# 这是ToT架构的"状态评估器"(State Evaluator)
# 负责剪除无效路径和循环路径
def prune_paths(state: ToTState) -> Dict[str, Any]:
    """
    路径剪枝节点 - 移除无效和循环路径
    剪枝策略:
    1. 循环检测：如果状态重复出现，剪除（防止无限循环）
    2. 有效性检查：get_possible_moves已确保合法性，此处可添加额外检查

    参数:state: 当前ToT状态，包含待剪枝的路径
    返回:dict: 包含剪枝后路径的状态更新
    """
    console.print("--- Pruning Paths ---")
    pruned_paths = []
```

```
    for path in state['active_paths']:
        # 循环检测：如果最后一个状态在路径中已经出现过，说明形成了循环
        if path[-1] in path[:-1]:
            continue  # 发现循环，剪除该路径

        # get_possible_moves函数已确保移动的合法性
        pruned_paths.append(path)

    console.print(f"[green]Pruned down to {len(pruned_paths)} valid, non-
cyclical paths.[/green]")
    return {"active_paths": pruned_paths}

# ==================== 解检测条件节点 ====================
def check_for_solution(state: ToTState) -> str:
    """
    解检测节点 - 判断是否找到解决方案
    遍历所有活跃路径，检查是否有路径达到目标状态

    参数:state: 当前ToT状态
    返回:str: 路由决策
        - "solution_found": 找到解，终止搜索
        - "continue_search": 未找到解，继续扩展
    """
    for path in state['active_paths']:
        # 检查路径的最后一个状态是否为目标状态
        if path[-1].is_goal():
            console.print("[bold green]Solution Found![/bold green]")
            # 副作用：将找到的解存储到状态中
            state['solution'] = path
            return "solution_found"
    return "continue_search"

# ==================== 思维树图构建 ====================
# 构建ToT架构的计算图，实现迭代深度优先的树搜索

# 初始化状态图构建器
workflow = StateGraph(ToTState)

# 添加三个核心节点
workflow.add_node("initialize", initialize_search)  # 初始化节点：创建根节点
workflow.add_node("expand", expand_paths)           # 扩展节点：生成所有可能下
一状态
workflow.add_node("prune", prune_paths)             # 剪枝节点：移除无效路径

# 定义执行流程
workflow.set_entry_point("initialize")              # 入口点：初始化搜索
workflow.add_edge("initialize", "expand")           # 初始化完成后扩展
workflow.add_edge("expand", "prune")                # 扩展完成后剪枝

# 条件边：剪枝后检查是否找到解
workflow.add_conditional_edges(
    "prune",
    check_for_solution,
    {
```

```
        "solution_found": END,      # 找到解，终止
        "continue_search": "expand" # 未找到，继续扩展
    }
)

# ==================== 图编译与应用初始化 ====================
# 将构建的ToT图编译为可执行的应用实例
tot_agent = workflow.compile()

print("Tree-of-Thoughts agent graph compiled successfully.")
```

阶段 3：演示与分析

在谜题上运行思维树智能体。把它**系统化**的方法与**简单的、单次通过的思维链请求**进行对比，以突出两者在鲁棒性方面的差异。

```
# ==================== 问题定义 ====================
# 定义狼-羊-白菜谜题的完整描述，该问题将被传递给ToT智能体和CoT智能体进行求解
problem = """A farmer wants to cross a river with a wolf, a goat, and a
cabbage.
The boat can only carry the farmer and one other item.
The farmer cannot leave the wolf alone with the goat, nor the goat alone
with the cabbage.
How can the farmer get everyone across safely?"""

# ==================== 运行思维树智能体 ====================
# 执行ToT架构进行问题求解
# ToT会进行多路径探索，通过扩展-剪枝循环找到解
console.print("--- 🌳 Running Tree-of-Thoughts Agent ---")

# 配置递归限制，防止无限循环
# 狼-羊-白菜谜题的最优解需要7步，设置15步限制足够搜索
config = {"recursion_limit": 15}

# 调用ToT智能体
# 智能体会自动执行流程：initialize → expand → prune → (检查解) → 循环直到找到解或
达到限制
final_state = tot_agent.invoke({"problem_description": problem},
config=config)

# ==================== 输出ToT求解结果 ====================
console.print("\n--- ✅ ToT Agent Solution ---")

# 检查是否找到解
if final_state.get('solution'):
    solution_path = final_state['solution']  # 获取完整的状态序列

    # 使用Rich Tree组件以树形结构可视化解路径
    tree = Tree("[bold magenta]Wolf, Goat, and Cabbage Solution Path[/bold
magenta]")
```

```
    # 遍历解路径中的所有状态，输出每一步的描述
    # 枚举从1开始，便于用户理解步骤顺序
    for i, state in enumerate(solution_path):
        tree.add(f"[green]{i+1}.[/green] {state.move_description}")

    console.print(tree)
else:
    # 如果未找到解（如达到递归限制），输出错误信息
    console.print("[bold red]No solution found within the step limit.[/bold red]")

# ==================== 运行思维链智能体 ====================
# 对比实验：使用传统的链式思维(Chain-of-Thought)求解同一问题
# 这是单路径线性推理，不会探索多种可能性
console.print("\n--- 🤔 Running Simple Chain-of-Thought Agent ---")

# 构建CoT提示模板
cot_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a world-class logic puzzle solver. Provide a step-by-step solution to the user's puzzle."),
    ("human", "{problem}")
])

# 创建CoT处理链
cot_chain = cot_prompt | llm

# 执行CoT推理
cot_result = cot_chain.invoke({"problem": problem}).content

# 以Markdown格式输出CoT结果
console.print(Markdown(cot_result))
```

结果分析

两种方法之间的差异是显著的：

- **思维链（CoT）：** 这种方法**依赖于大语言模型的预训练知识来回忆解决方案**。对于像这样经典、广为人知的问题，一个强大的大语言模型通常能一次性给出正确答案。然而，一旦它犯了一个错误，就**没有自我修正的机制**。对于新颖或更复杂的问题，失败的可能性则要高得多。**其正确性依赖于“回忆”，而非可验证的推理**。

- **思维树（ToT）：** 这个智能体通过**系统化的、可验证的搜索**发现了解决方案。它不仅仅是回忆答案，而是**构建答案**。从日志中看到这个过程：**扩展路径，然后剪除那些进入死胡同或循环的分支**。即使引导扩展的大语言模型在某条分支上做出了**次优选择**，智能体仍能继续探索其他**更有前景的分支**。这种方法更为鲁棒和可信，因为其最终解决方案由定义的环境规则被保证是有效的。

思维树智能体的成功并非基于运气或记忆，而是基于**其搜索算法的可靠性**。这使得它成为那些对**高可靠性和规划能力有要求**的任务中，一种远为优越的方法。

结论

在本节笔记本中，我们实现了一个**思维树**智能体来解决一个经典的逻辑谜题。我们证明了，**通过将问题转化为状态空间并对其进行系统化搜索**，智能体能够达到单次通过推理方法无法企及的鲁棒性和准确性水平。

思维树的核心组件——**思维生成（扩展）**、**状态评估（剪枝）** 和**搜索**——共同构成了一个强大的框架，用于处理**复杂的规划与推理任务**。尽管其计算成本更高，但这种权衡换来了可靠性和问题解决能力的显著提升。这种架构是构建能够**深思熟虑地推理**并为具有挑战性的**多步骤问题**找到解决方案的智能体的关键一步。